

Respuesta de Active-IA a AI-Native

Sobre los §4.1 y §4.2 de su documento del 27/08.

De: equipo de Active-IA (api.active-ia.com)

Para: equipo de AI-Native (tutor.active-ia.com)

Fecha: 27 de agosto de 2026

§4.1 — El detalle por caso: nos adaptamos nosotros

Decisión: no cambien nada. El nombre salida_obtenida **queda.** Nos ofrecieron renombrarlo a obtenido en una línea; preferimos que no.

- 1 **Su nombre es mejor.** obtenido no dice obtenido *qué*. salida_obtenida sí.
- 2 **Ustedes tienen tests escritos contra ese nombre.** Tocarlos para que coincidan con un documento que estaba mal es arreglar el artefacto equivocado.
- 3 **El costo es asimétrico y cae de nuestro lado.** Su servicio ya corre; el nuestro todavía no recibió una entrega real.

Tenían razón en que era el punto más accionable

Habíamos armado el parser leyendo su documento, así que esperábamos {id, paso, entrada, esperado, obtenido}. Lo que pasó no fue un error visible: **Pydantic descarta en silencio los campos que no conoce.** No hubo excepción, no hubo log, no hubo validación fallida. salida_obtenida se perdía y obtenido quedaba en None.

El efecto concreto: el motor recibía la sección de RESULTADO DE EJECUCIÓN sin la salida real del alumno — que es exactamente el hecho por el que esa sección existe. Constatava que el caso falló y no podía explicar por qué.

Corregido en el commit cca572d de la rama feat/integracion-ai-native. Además del arreglo, agregamos un test que **parte del payload real de su** _mapear y verifica que la salida sobrevive schema → prompt.

Vale la pena decir por qué, porque es la lección del episodio: cada capa nuestra se probaba con su propio diccionario inventado, y las dos estaban de acuerdo entre ellas y equivocadas respecto de ustedes. Dos tests verdes tapando el agujero. El único test que puede cazar esto es el que arranca del payload del otro lado.

entrada y esperado: los eliminamos, no los dejamos opcionales

Coincidimos con el motivo que dieron. Y agregamos uno: un campo opcional que nadie llena es peor que ninguno, porque el próximo que lea el schema va a creer que ahí hay un dato.

es_publico: lo usamos, y filtramos en el código

Un caso oculto ahora aporta su veredicto al prompt (cuenta para la nota) pero **su nombre y su salida no salen de nuestro backend.** El motor ve esto y nada más:

- [t9] FALLÓ (caso oculto)

El detalle que importa: **filtramos en el código, no pidiéndoselo al motor en el prompt.** Pedirle que no mencione algo que igual le mostramos es confiar en que obedezca, y de este motor ya tenemos medido que

no siempre honra las reglas que se le declaran — es el bug 2 que les reportamos, donde la rúbrica pedía una penalización del 30% y aplicó 0%.

Al prompt sí va una instrucción, pero contra el otro riesgo, que el filtro no cubre: que el motor, viendo un caso sin nombre ni salida, **se invente** de qué se trataba para redactar la devolución. Se le pide que diga que hay casos adicionales que no pasaron, sin describirlos.

Lo que revisamos y NO había que tocar

Fuimos a buscar un segundo desajuste en el nivel superior (`compila / error_compilacion / total / pasados / casos`) y **no existe**. Su `correccion_ejecutor.py` ya remapea `passed` → `pasados` y descarta `failed` antes del POST. Lo decimos explícitamente para que nadie «arregle» algo que está bien.

Un pendiente que les proponemos, con una pregunta

Hoy el prompt le muestra al motor **qué salió** pero no **contra qué se comparaba**. Y ustedes hacen bien en no mandar `salida_esperada`: la definición del caso se la dimos nosotros al publicar el TP, repetírmola sería redundante.

Podemos correlacionar por `id` contra la definición que ya tenemos persistida. Y es seguro por construcción: nuestro schema de `TestCase` **rechaza** un caso oculto que traiga `salida_esperada` o `asercion` — no lo limpia en silencio, falla al publicar. Así que un caso oculto no tiene qué filtrar.

La pregunta: ¿el `id` que mandan en el resultado es el mismo `id` que recibieron en la definición del TP? Vimos en su `_mapear` un `c.get("id")` or `c.get("test_id")`, y si ese segundo camino produce un `id` propio del runner, la correlación no cierra. Con un sí, lo implementamos; mueve el presupuesto de tokens que habíamos medido, así que va como paso propio.

§4.2 — El campo de la nota

Es nota. Bajen la cascada y dejen ese solo. No existe ni `nota_100`, ni `nota_final`, ni `calificacion`.

Tenían razón en que esa cascada es el tipo de cosa que funciona hasta que un día devuelve el campo equivocado.

La escala: 0–100, y no por convención

`nota` está siempre sobre 100. No es un acuerdo tácito, está forzado en dos lugares:

- El `puntaje_maximo` de una rúbrica tiene un validador que **exige exactamente 100**; cualquier otro valor es rechazado al crearla.
- Cuando ustedes publican un TP con pesos arbitrarios por criterio, los normalizamos a que sumen exactamente 100 (reparto por resto mayor, sin perder puntos por redondeo).

Así que `nota_100` era una buena intuición sobre la escala. Solo estaba mal el nombre.

El detalle que les va a morder y que no preguntaron

`nota` **viaja como string JSON, no como número**:

```
{ "correccion_id": 10, "nota": "85.50", "criterios": [],  
  "criterios_sin_ejecucion": [] }
```

Es "85.50", con comillas. Es el comportamiento por defecto de Pydantic v2 para `Decimal` y lo dejamos a propósito: una nota no debería pasar por un `float` en ningún tramo del camino.

Lo señalamos porque es exactamente el mismo mecanismo que el §4.1. `float("85.50")` funciona, así que un parser puede andar por accidente durante meses y romperse el día que alguien agregue una comparación de tipos o un `if not nota`. **Casteen explícitamente.**

Si prefieren que salga como número, díganlo y lo cambiamos — pero nuestra recomendación es que no, y que el casteo quede visible de su lado.

Confirmado lo que dieron por confirmado

- `criterios_sin_ejecucion` viene como lista aparte. Nos parece muy bien cómo lo resolvieron en el panel: «sin verificar» en lugar del puntaje es exactamente la lectura correcta. Un cero ahí significa que ninguna corrida respalda el criterio, no que el alumno no lo haya hecho.
- La respuesta **no trae nota agregada del TP**. El promedio ponderado es de ustedes.

Resumen de qué tiene que hacer cada uno

Quién	Qué
Active-IA	Hecho: parser del caso alineado (cca572d), <code>es_publico</code> respetado, test de extremo a extremo contra su payload real.
AI-Native	Bajar la cascada de la nota y dejar nota. Castear el string.
AI-Native	Responder si el <code>id</code> del resultado es el mismo que el de la definición del TP.
Ambos	Nada más de §4.1: el nivel superior ya estaba bien de los dos lados.